

CyberAudit

Training Modules API

5 cybersecurity modules - Progress tracking per user - Idempotent start/complete

Project	CyberAudit Platform
Module	apps/training/
Models	TrainingModule + TrainingProgress
5 module slugs	anti-phishing / mfa / wifi-vpn / data-backup / incident-response
Progress statuses	to_start (virtual) / in_progress / completed
UniqueConstraint	UniqueConstraint(['user','module']) -- 1 row per pair
Idempotent start	POST /start/ twice keeps first started_at unchanged
Skip-ahead complete	POST /complete/ without prior start auto-fills started_at
user_status	SerializerMethodField injecting per-user progress
Tests	14 tests across 4 classes
Date	June 2026 / Juin 2026

1) Data Models

1) Modèles de données

The training app uses two models: TrainingModule (fixed catalogue of cybersecurity awareness modules) and TrainingProgress (per-user, per-module progress enforced unique by a DB constraint). `get_or_create()` prevents duplicates in views.

L'app training utilise deux modèles : TrainingModule (catalogue fixe) et TrainingProgress (progression par utilisateur et par module, garantie unique par contrainte DB). `get_or_create()` prévient les doublons.

TrainingModule

```
class TrainingModule(models.Model):
    class Level(models.TextChoices):
        BEGINNER = 'beginner', 'Debutant'
        INTERMEDIATE = 'intermediate', 'Intermediaire'
        ADVANCED = 'advanced', 'Avance'

    slug = models.SlugField(unique=True, max_length=80)
    title = models.CharField(max_length=200)
    description = models.TextField(blank=True)
    content_md = models.TextField(blank=True) # Markdown -- detail view only
    duration_min = models.PositiveSmallIntegerField(default=10)
    level = models.CharField(choices=Level.choices, default=Level.BEGINNER)
    published_at = models.DateTimeField(null=True, blank=True) # NULL = draft
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'training_modules'
        ordering = ['id']
```

TrainingModule field notes

Field	Type	Purpose EN / FR
slug	SlugField (unique)	URL-safe identifier ('anti-phishing'). Frontend routing key. Identifiant URL-safe. Clé de routage frontend.
content_md	TextField	Full Markdown course, only in detail serializer (bandwidth saving). Cours Markdown complet, uniquement dans detail serializer.
duration_min	PositiveSmallInt	Estimated completion time shown on UI card. Temps de completion estime affiche sur la carte UI.
level	CharField (choices)	beginner / intermediate / advanced. Shown as badge in UI. beginner / intermediate / advanced. Badge dans l'UI.

published_at	DateTimeField (nullable)	NULL = draft hidden from all API endpoints. Filter: <code>isnull=False</code>. NULL = brouillon cache. Filtre : <code>published_at__isnull=False</code>.
---------------------	--------------------------	--

TrainingProgress

```
class TrainingProgress(models.Model):
    class Status(models.TextChoices):
        TO_START = 'to_start', 'A commencer'
        IN_PROGRESS = 'in_progress', 'En cours'
        COMPLETED = 'completed', 'Termine'

    user = models.ForeignKey(AUTH_USER_MODEL, on_delete=CASCADE)
    module = models.ForeignKey(TrainingModule, on_delete=CASCADE)
    status = models.CharField(choices=Status.choices,
default=Status.TO_START, db_index=True)
    started_at = models.DateTimeField(null=True, blank=True)
    completed_at = models.DateTimeField(null=True, blank=True)

    class Meta:
        db_table = 'training_progress'
        constraints = [
            models.UniqueConstraint(fields=['user', 'module'],
name='uniq_user_module'),
        ]
        ordering = ['-updated_at']
```

UniqueConstraint	DB-level guarantee: second insert for same (user, module) raises IntegrityError. Views use <code>get_or_create()</code>. Garantie niveau DB : deuxième insert pour (user, module) identique lève IntegrityError.
on_delete=CASCADE	Both FKs: deleting a user removes their progress; deleting a module removes all its entries. Les deux FK : supprimer un user ou un module supprime les entrées de progression associées.
Default TO_START	<code>user_status</code> returns <code>TO_START</code> without a DB row. Row only created on first start/complete action. <code>user_status</code> retourne <code>TO_START</code> sans ligne DB. Ligne créée au premier start/complete.
update_fields	start/complete use <code>save(update_fields=...)</code> -- only modified columns written. start/complete utilisent <code>save(update_fields=...)</code> -- colonnes modifiées uniquement.

2) The 5 Cybersecurity Modules

2) Les 5 modules de sensibilisation cyber

Five modules seeded at deployment (fixtures/data migration). All published (published_at IS NOT NULL). They cover the most common threats SMBs face.

Cinq modules seedés au deployment. Tous publiés. Ils couvrent les menaces les plus courantes des PME.

Slug	Title	Level	Topic EN / FR
anti-phishing	Anti-Phishing	beginner	Recognizing phishing emails, spoofed senders, malicious links. Reconnaître emails phishing, expéditeurs usurpés, liens malveillants.
mfa	Multi-Factor Auth	beginner	Why passwords alone are insufficient. TOTP, hardware keys, push notifications. Pourquoi les mots de passe seuls sont insuffisants. TOTP, clés matérielles.
wifi-vpn	Secure Wi-Fi & VPN	intermediate	Risks of public Wi-Fi, WPA3 basics, VPN for remote work. Risques Wi-Fi public, WPA3, usage VPN en télétravail.
data-backup	Data Backup Strategy	intermediate	3-2-1 backup rule, RPO/RTO concepts, ransomware resilience. Règle 3-2-1, RPO/RTO, résilience au ransomware.
incident-response	Incident Response	advanced	Steps when breach occurs: contain, eradicate, recover, report. Étapes en cas de compromission : contenir, éradiquer, récupérer, signaler.

3) Serializers : 3 classes

3) Sérialiseurs : 3 classes

Three serializers with clear inheritance: TrainingModuleListSerializer (no Markdown, with user_status) is extended by TrainingModuleDetailSerializer (adds content_md). TrainingProgressSerializer is the action response.

Trois sérialiseurs avec héritage clair. TrainingModuleDetailSerializer étend le sérialiseur liste en ajoutant content_md. TrainingProgressSerializer est retourné par start/complete.

Serializer	Used by	Fields / purpose
TrainingModuleListSerializer	GET /modules/	8 fields: id, slug, title, description, duration_min, level, published_at, user_status. No content_md. 8 champs: id, slug, title, description, duration_min, level, published_at, user_status.. Pas de content_md.
TrainingModuleDetailSerializer	GET /modules/{id}/	Inherits list serializer + adds content_md. 9 fields. class Meta(ListSerializer.Meta). Herite + ajoute content_md. 9 champs. class Meta(ListSerializer.Meta).
TrainingProgressSerializer	POST /start/ POST /complete/	3 read-only: status, started_at, completed_at. Action response body. 3 read_only, started_at, completed_at. Corps de réponse des actions.

user_status SerializerMethodField

```
def get_user_status(self, obj):
    request = self.context.get('request')
    if not request or not request.user.is_authenticated:
        return None
    progress = TrainingProgress.objects.filter(
        user=request.user, module=obj,
    ).first()
    # No DB row? Return 'to_start' without creating anything
    return progress.status if progress else TrainingProgress.Status.TO_START

# TrainingModuleDetailSerializer inherits via:
class TrainingModuleDetailSerializer(TrainingModuleListSerializer):
    class Meta(TrainingModuleListSerializer.Meta):
        fields = TrainingModuleListSerializer.Meta.fields + ['content_md']
        read_only_fields = fields
```

4)Views & Endpoint Logic

4) Vues et logique des endpoints

All 4 endpoints require `IsAuthenticated`. List/detail filter to published modules only via `get_queryset()`. start and complete use `get_or_create()` + conditional save for idempotence.

Les 4 endpoints requierent `IsAuthenticated`. Liste/detail filtrent `published_at__isnull=False`. start et complete utilisent `get_or_create()` + save conditionnel pour l'idempotence.

`_published_module_or_404()` helper

```
def _published_module_or_404(pk):
    try:
        return TrainingModule.objects.get(pk=pk, published_at__isnull=False)
    except TrainingModule.DoesNotExist:
        raise Http404 from None
    # 404 for: unknown pk AND draft module
```

StartModuleView -- idempotent

```
class StartModuleView(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request, pk):
        module = _published_module_or_404(pk)
        progress, _ = TrainingProgress.objects.get_or_create(
            user=request.user, module=module,
        )
        # IDEMPOTENT: only update if still to_start
        if progress.status == TrainingProgress.Status.TO_START:
            progress.status = TrainingProgress.Status.IN_PROGRESS
            progress.started_at = timezone.now()
            progress.save(update_fields=['status', 'started_at'])
        # Already in_progress or completed -> no change
        return Response(TrainingProgressSerializer(progress).data) # 200
```

CompleteModuleView -- skip-ahead

```
class CompleteModuleView(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request, pk):
        module = _published_module_or_404(pk)
        progress, _ = TrainingProgress.objects.get_or_create(
            user=request.user, module=module,
        )
        now = timezone.now()
        progress.status = TrainingProgress.Status.COMPLETED
        progress.completed_at = now
        if not progress.started_at: # SKIP-AHEAD
            progress.started_at = now # auto-fill if bypassed start
        progress.save(update_fields=['status', 'started_at', 'completed_at'])
        return Response(TrainingProgressSerializer(progress).data) # 200
```

5) URL Routing & Endpoint Reference

5) Routage URL et référence des endpoints

```
# training/urls.py -- prefixed with /api/ in config/urls.py
urlpatterns = [
    path('training/modules/', TrainingModuleListView.as_view(),
        name='training-list'),
    path('training/modules/<int:pk>/', TrainingModuleDetailView.as_view(),
        name='training-detail'),
    path('training/modules/<int:pk>/start/', StartModuleView.as_view(),
        name='training-start'),
    path('training/modules/<int:pk>/complete/', CompleteModuleView.as_view(),
        name='training-complete'),
]
```

URL	Method	Auth	Status	Description EN / FR
GET /api/training/modules/	GET	Bearer	200, 401	Paginated published modules with user_status injected. Modules publiés paginés avec user_status injectée.
GET /api/training/modules/{id}/	GET	Bearer	200, 401, 404	Full detail + content_md. 404 for draft or unknown. Detail complet + content_md. 404 pour brouillon ou inconnu.
POST /api/training/modules/{id}/start/	POST	Bearer	200, 401, 404	Idempotent start. Returns {status, started_at, completed_at}. Démarrage idempotent. Retourne {status, started_at, completed_at}.
POST /api/training/modules/{id}/complete/	POST	Bearer	200, 401, 404	Mark completed. Auto-fills started_at if not set. Marquer terminé. Auto-remplit started_at si absent.

Responses examples / Exemples de réponses

```
// GET /modules/ -- list item:
{ 'slug': 'anti-phishing', 'level': 'beginner', 'duration_min': 15,
  'user_status': 'to_start' } // injected per-user

// POST /modules/1/start/ -- first call:
{ 'status': 'in_progress', 'started_at': '2026-06-17T19:54:00Z', 'completed_at':
null }

// POST /modules/1/start/ -- second call (idempotent):
{ 'status': 'in_progress', 'started_at': '2026-06-17T19:54:00Z', 'completed_at':
null }
// started_at UNCHANGED

// POST /modules/1/complete/ -- skip-ahead (no prior start):
{ 'status': 'completed', 'started_at': '2026-06-17T20:00:00Z', 'completed_at':
'2026-06-17T20:00:00Z' }
```


6) Progress Lifecycle : State Machine

6) Cycle de vie : Machine d'état

Status	DB row?	Who sets it	Behaviour EN / FR
to_start	No (virtual)	Default	user_status returns 'to_start' when no row exists. No DB write. user_status retourne 'to_start' sans ligne DB.
in_progress	Yes	POST /start/	Row created with status='in_progress', started_at=now(). Idempotent: 2nd call no-op. Ligne créée. Idempotent : 2eme appel sans effet.
completed	Yes	POST /complete/	status='completed', completed_at=now(). started_at auto-filled if null. status='completed', started_at auto-rempli si null.

to_start -> in_progress	POST /start/ transitions only if status == TO_START. Creates DB row. POST /start/ transite uniquement si status == TO_START. Crée une ligne de base de données.
to_start -> completed	Skip-ahead: POST /complete/ without prior /start/. Row created as completed, started_at=now(). Saut : POST /complete/ sans /start/. Ligne créée complete, started_at=now().
in_progress -> completed	Normal flow: /start/ then /complete/. started_at preserved. Flux normal: started_at preserve du start.
completed -> completed	Re-completing overwrites completed_at. No regression. Re-completing écrase completed_at. Pas de regression.
in_progress -> in_progress	POST /start/ no-op. Returns current state unchanged. POST /start/ sans effet. Retourne l'état inchangé.

7) Tests : 14 tests across 4 classes

7) Tests : 14 tests en 4 classes

Validates catalogue seeding, DB uniqueness constraint, publication filtering, default user_status, idempotent start, skip-ahead complete, and 404 handling.

Valide : seeding catalogue, unicity DB, filtrage publication, user_status par défaut, start idempotent, complete direct, gestion 404.

Class	Test	HTTP	Assertion EN / FR
TestTrainingModels	test_five_modules_seeded	200	5 modules, slugs match expected set exactly. 5 modules, slugs correspondent exactement.
TestTrainingModels	test_all_seeded_modules_are_published	200	0 modules with published_at IS NULL. 0 modules avec published_at IS NULL.
TestTrainingModels	test_unique_user_module_constraint	401	2nd create for same (user, module) raises IntegrityError. 2eme create pour (user, module) identique leve IntegrityError.
TestTrainingList	test_anonymous_returns_401	401	Unauthenticated GET /modules/ returns 401. GET non authentifié retourne 401.
TestTrainingList	test_authenticated_sees_5_modules	200	auth_client gets exactly 5 paginated results. auth_client obtient exactement 5 résultats paginés.
TestTrainingList	test_unpublished_modules_are_filtered_out	200	Draft module created still 5 results, draft slug absent. Module brouillon créé toujours 5 résultats.

TestTrainingList	test_user_status_defaults_to_to_start	200	All 5 modules return user_status=='to_start' before any action. Les 5 modules retournent 'to_start' avant toute action.
TestStartModule	test_start_creates_progress_in_progress	200	status=='in_progress', started_at not None. Confirmed refresh_from_db(). status=='in_progress', confirme via refresh_from_db().
TestStartModule	test_start_is_idempotent	200	Two /start/ calls: r2.started_at == r1.started_at (not reset). Deux /start/ appels : r2.started_at == r1.started_at (non réinitialisé).
TestStartModule	test_start_on_unknown_module_returns_404	404	pk=99999 raises Http404. pk=99999 lève Http404.
TestCompleteModule	test_complete_module_already_started	200	Start then complete: status=='completed', completed_at not None. Start puis complete : completed_at non null.
TestCompleteModule	test_complete_creates_progress_if_missing	200	Complete without start: status=='completed', started_at auto-filled. Complete sans start : started_at auto-rempli.

8. Security & Design Summary

8. Récapitulatif sécurité et conception

Feature	Detail EN / FR
Auth required	IsAuthenticated on all 4 endpoints. No public training content. IsAuthenticated sur les 4 endpoints. Pas de contenu public.
Published filter	published_at__isnull=False everywhere. Draft modules never exposed via API. Filtrage partout. Modules brouillons jamais exposes.
DB-level uniqueness	published_at__isnull= UniqueConstraint on (user, module). get_or_create() handles concurrency. published_at__isnull= UniqueConstraint sur (user, module). get_or_create() gère la concurrence.
Idempotent /start/	POST /start/ on already-started/completed returns current state. No regression. POST /start/ on already-started/completed retourne l'état actuel inchangé. Pas de régression.
content_md isolation	Markdown only in detail serializer. List never returns large text blobs. Markdown uniquement dans le sérialiseur détail. La méthode List ne renvoie jamais de gros blocs de texte.
update_fields	Both actions save only modified columns, no full-row UPDATE. Les deux actions sauvegardent uniquement les colonnes modifiées, pas de mise à jour de ligne complète
Per-user isolation	user_status per request.user. Progress never leaks between users. user_status par request.user. La progression ne fuit jamais entre utilisateurs.
CASCADE delete	Both FKs cascade. Deleting user or module removes related progress. CASCADE sur les deux FK. La suppression d'un utilisateur ou d'un module entraîne la suppression de la progression associée.